

TECNOLÓGICO DE COSTA RICA
Escuela de Computación
Principios de Sistemas Operativos

Segundo Proyecto

jqindex — Indexador y Buscador de Archivos JSON

Curso:	Principios de Sistemas Operativos
Proyecto:	Segundo Proyecto
Entregable:	Librería libjsonindex + CLI jqindex
Lenguaje:	C / C++

Tabla de Contenidos

1.	Introducción	3
2.	Análisis del Problema	3
3.	Solución Propuesta	4
4.	Explicación del Parser JSON	5
5.	Formato del Índice (.jnx)	7
6.	Búsquedas con Expresiones Regulares	8
7.	Estructura del Proyecto	9
8.	Problemas Encontrados	10
9.	Pruebas Realizadas	11
10.	Conclusiones	13

1. Introducción

Este proyecto consiste en desarrollar una librería en C y un programa de línea de comandos llamado **jindex**, que permite indexar archivos JSON grandes y realizar búsquedas eficientes sobre ellos usando expresiones regulares, sin necesidad de parsear el archivo completo en cada consulta.

La motivación del proyecto viene del concepto de archivos ISAM (Indexed Sequential Access Method), que los sistemas operativos usaban históricamente para acceder eficientemente a registros en disco. La idea es trasladar ese concepto al mundo actual donde los datos semiestructurados en formato JSON son muy comunes.

El programa implementa dos operaciones principales: **build**, que analiza el JSON y genera un archivo de índice con extensión **.jnx**; y **search**, que aplica una expresión regular sobre el índice y recupera los fragmentos correspondientes del archivo original usando **fseek**.

2. Análisis del Problema

2.1 El problema de buscar en JSON grandes

Cuando un archivo JSON es muy grande, cargarlo completo en memoria para buscar un valor específico es ineficiente. La solución es crear un índice separado que mapee cada ruta del JSON a su posición en bytes dentro del archivo original. Así, una búsqueda solo necesita leer el índice (que es pequeño) y luego saltar directamente al fragmento relevante con **fseek**.

2.2 ¿Qué es un path JSON?

Un path es la ruta absoluta a un valor dentro de un JSON, similar a JSON Pointer. Por ejemplo, dado el JSON:

```
{
  "usuarios": [
    { "id": 1, "nombre": "Ana" }
  ]
}
```

El path del campo *nombre* es **/usuarios/0/nombre**. Los arrays usan índices numéricos empezando desde 0.

2.3 Restricciones del proyecto

El enunciado establece que no se pueden usar librerías externas para parsear JSON. Hay que escribir un parser propio, asumiendo que el archivo está bien formado. La búsqueda debe implementarse con **regex.h**, la librería estándar de POSIX, y la recuperación de fragmentos con **fseek** de Unix.

3. Solución Propuesta

3.1 Arquitectura general

Se optó por una arquitectura modular simple con cuatro componentes, sin clases ni patrones complejos. Cada módulo tiene una responsabilidad clara:

Módulo	Archivo	Responsabilidad
Parser JSON	json_parser.c	Recorre el JSON y genera la lista de paths con offsets
Indexador	indexer.c	Escribe y lee el archivo .jnx en texto plano
Buscador	searcher.c	Aplica regex sobre el índice y reconstruye la salida JSON
CLI	jqindex.cpp	Maneja los argumentos y coordina los otros módulos

3.2 Flujo de ejecución

Comando build:

```
jqindex build datos.json
```

1. Leer datos.json completo a memoria
2. parse_json() -> genera lista de (path, inicio, fin)
3. write_index() -> escribe datos.jnx
4. Reportar cuantas entradas se generaron

Comando search:

```
jqindex search datos.json "$/usuarios/[0-9]+/nombre"
```

1. Verificar que datos.jnx existe
2. read_index() -> cargar el .jnx a memoria
3. Preparar regex (\$ inicial -> ancla ^)
4. Para cada entrada del indice:
 regexec(regex, path) -> si coincide:
 fseek(datos.json, inicio)
 leer (fin - inicio + 1) bytes
5. Imprimir resultado como array JSON o valor unico

3.3 Por qué se eligió esta solución

Se priorizó la simplicidad sobre la elegancia. El parser es recursivo porque es más fácil de entender y depurar que una versión iterativa con stack manual. El índice es texto plano (no binario) porque cualquiera puede abrirlo en un editor y verificar que está correcto, lo que fue útil durante el desarrollo. No se usaron clases de C++ porque los structs de C son suficientes para este problema.

4. Explicación del Parser JSON

4.1 Estrategia general

El parser carga el archivo completo en un buffer de caracteres y lo recorre usando un índice entero **pos**. La función principal es **parse_value()**, que detecta el tipo del carácter actual y decide cómo procesarlo:

```
parse_value(buf, &pos, path_actual, resultado)
'{' -> parse_object()    // objeto JSON
'[' -> parse_array()     // array JSON
'"' -> string            // buscar la " de cierre
'0-9' o '-' -> numero    // avanzar mientras sea dígito
'true/false/null' -> literal de longitud fija
```

4.2 Cómo se construye el path

El path se pasa como parámetro a cada llamada recursiva y se va extendiendo. Para objetos se agrega la clave: **path + "/" + clave**. Para arrays se agrega el índice numérico: **path + "/" + numero**. La raíz empieza con path vacío, y el primer nivel queda como **/clave**.

```
// En parse_object:
snprintf(new_path, sizeof(new_path), "%s/%s", current_path, key);

// En parse_array:
snprintf(new_path, sizeof(new_path), "%s/%d", current_path, index);
```

4.3 Cálculo de offsets

El **inicio** es la posición de **pos** justo antes de entrar al valor (el {, [, ", o primer dígito). El **fin** se calcula de forma distinta según el tipo:

Tipo de valor	Cómo se calcula el fin
Objeto {}	Contar llaves balanceadas hasta depth == 0
Array []	Contar corchetes balanceados hasta depth == 0
String	Avanzar hasta " de cierre, respetando escapes \"
Número	Avanzar mientras sea dígito, punto, e, signo
true / false	inicio + 3 / inicio + 4 (longitud fija)
null	inicio + 3 (longitud fija)

4.4 Detección del fin de bloques (objetos y arrays)

La función **find_block_end()** cuenta la profundidad de anidamiento. Empieza en depth=1 (ya vimos el abre). Cada { o [incrementa, cada } o] decrementa. Cuando llega a 0, encontramos el cierre:

```
depth = 1
pos++ // saltamos el { o [ inicial
mientras pos < size y depth > 0:
    si buf[pos] == '"': saltar el string completo
    si buf[pos] == open_char: depth++
    si buf[pos] == close_char: depth--
    pos++
retornar pos - 1 // el } o ] de cierre
```

Nota: es importante saltarse los strings completos dentro del bloque, porque pueden contener caracteres { o } que no son delimitadores reales.

4.5 Limitaciones del parser

Como el enunciado permite un parser 'quick and dirty', hay algunas limitaciones conocidas: no se manejan escapes Unicode (`\u0022`), no se valida que el JSON esté bien formado, y números con notación científica compleja podrían calcular el fin incorrectamente en casos extremos. Para el alcance del proyecto estas limitaciones no afectan los casos de prueba normales.

5. Formato del Índice (.jnx)

5.1 Estructura del archivo

El archivo **.jnx** es texto plano, una entrada por línea. El formato de cada línea es:

```
path|inicio|fin
```

El separador **|** se eligió porque no aparece en paths JSON válidos. Los campos son: **path** (ruta absoluta del elemento), **inicio** (offset en bytes del primer carácter del valor) y **fin** (offset del último carácter, inclusive). Ambos offsets son absolutos desde el inicio del archivo, aptos para usar directamente con **fseek(..., SEEK_SET)**.

5.2 Ejemplo real generado

Para el archivo **datos.json** del enunciado, el índice generado es:

```
/|0|175
/usuarios|14|109
/usuarios/0|16|59
/usuarios/0/id|24|24
/usuarios/0/nombre|37|41
/usuarios/0/activo|54|57
/usuarios/1|62|107
/usuarios/1/id|70|70
/usuarios/1/nombre|83|88
/usuarios/1/activo|101|105
/config|122|173
/config/version|135|139
/config/opciones|154|171
/config/opciones/modo|164|169
```

5.3 Verificación manual de un offset

Para verificar que los offsets son correctos, se puede usar el siguiente comando en Linux, que lee exactamente los bytes del campo **nombre** del primer usuario (inicio=37, fin=41, longitud=5):

```
# Leer 5 bytes desde la posición 37 del archivo
dd if=tests/datos.json bs=1 skip=37 count=5 2>/dev/null
# Salida esperada: "Ana"
```

La entrada con path "/" representa el documento JSON completo.

6. Búsquedas con Expresiones Regulares

6.1 Preparación del patrón

El patrón llega del CLI con un **\$** inicial que representa la raíz (convención del proyecto). Antes de pasarlo a **regcomp()**, se transforma:

```
Entrada:  $/usuarios/[0-9]+/nombre
Salida:   ^/usuarios/[0-9]+/nombre$

// El $ inicial se reemplaza por ^ (ancla al inicio)
// Se agrega $ al final (ancla al fin)
// Esto obliga a match completo del path
```

Agregar el **\$** al final es importante: sin él, el patrón **/nombre** matchearía también con **/nombre_completo**.

6.2 Ejecución del match

```
regcomp(&regex, patron_preparado, REG_EXTENDED | REG_NOSUB);

for (int i = 0; i < indice.count; i++) {
    if (regexexec(&regex, indice.entries[i].path, 0, NULL, 0) == 0) {
        // coincidencia -> leer fragmento del JSON con fseek
    }
}
```

REG_EXTENDED habilita la sintaxis moderna de expresiones regulares (**[0-9]+**, **.**, *****, etc.). **REG_NOSUB** indica que no se necesitan los grupos capturados, lo que hace la operación un poco más rápida.

6.3 Recuperación de fragmentos con fseek

```
fseek(json_file, entrada.inicio, SEEK_SET);
long len = entrada.fin - entrada.inicio + 1;
char *frag = malloc(len + 1);
fread(frag, 1, len, json_file);
frag[len] = '\0';
```

6.4 Construcción de la salida

Si se encontró un solo resultado, se imprime el fragmento directamente. Si hay varios, se construye un array JSON concatenando con comas:

```
// Un resultado:
"1.0"

// Múltiples resultados:
["Ana", "Luis"]
[true, false]
[1, 2]
```


6.5 Ejemplos de expresiones soportadas

Expresión	Resultados obtenidos
<code>\$/usuarios/[0-9]+/nombre</code>	<code>["Ana", "Luis"]</code>
<code>\$/config/.*</code>	<code>["1.0", {"modo":"auto"}, "auto"]</code>
<code>\$/config/version</code>	<code>"1.0"</code>
<code>\$/usuarios/[0-9]+/activo</code>	<code>[true, false]</code>
<code>\$/usuarios/[0-9]+/id</code>	<code>[1, 2]</code>

7. Estructura del Proyecto

```
jqindex_project/
■■■ src/
■   ■■■ json_parser.h    <- tipos IndexEntry, IndexResult; firma de parse_json()
■   ■■■ json_parser.c    <- implementacion del parser recursivo
■   ■■■ indexer.h        <- firmas de write_index(), read_index(), get_jnx_path()
■   ■■■ indexer.c        <- lectura y escritura del archivo .jnx
■   ■■■ searcher.h       <- firma de search_index()
■   ■■■ searcher.c       <- logica de regex y reconstruccion de salida
■   ■■■ jqindex.cpp       <- main, manejo de argumentos CLI
■   ■■■ test_parser.c     <- prueba unitaria del parser
■   ■■■ test_indexer.c    <- prueba unitaria del indexador
■   ■■■ test_searcher.c   <- prueba unitaria del buscador
■■■ tests/
■   ■■■ datos.json        <- ejemplo del enunciado
■   ■■■ simple.json       <- JSON plano para prueba basica
■   ■■■ complejo.json     <- JSON con arrays anidados
■   ■■■ test.sh           <- script de 19 pruebas automaticas
■■■ Makefile
```

7.1 Compilación

El proyecto se compila con **make**. Los módulos **.c** se compilan a archivos **.o** con **gcc**, y el **.cpp** se compila y enlaza todo con **g++**. Esta separación fue necesaria para evitar problemas de name mangling entre C y C++.

```
make          # compila y genera el binario jqindex
make clean    # elimina binarios y archivos .jnx generados
```

7.2 La estructura IndexResult

La estructura central del proyecto es **IndexResult**, que contiene un arreglo de hasta 10 000 entradas. Cada entrada guarda el path (hasta 512 caracteres), el offset de inicio y el offset de fin. Esta estructura ocupa aproximadamente 5 MB, lo que causó un problema durante el desarrollo (ver sección 8).

```
typedef struct {
    char path[512];
    long inicio;
    long fin;
} IndexEntry;

typedef struct {
    IndexEntry entries[10000];
    int count;
} IndexResult;
```

8. Problemas Encontrados Durante el Desarrollo

8.1 Stack overflow con IndexResult

El primer problema serio fue un **Segmentation Fault** al declarar **IndexResult** como variable local en **main()**. La estructura ocupa cerca de 5 MB, y el stack típico de Linux es de 8 MB. Al declarar dos instancias (result y result2) en el mismo main para el test del indexador, el stack se desbordaba inmediatamente.

La solución fue declarar todas las instancias de **IndexResult** como **static**, lo que las ubica en el segmento de datos del proceso en lugar del stack. Esto se dejó documentado en el header con un comentario explícito.

```
// MAL - causa stack overflow:
int main() {
    IndexResult result;    // ~5MB en el stack
    IndexResult result2;   // otros ~5MB -> desbordamiento
}

// BIEN - vive en el segmento de datos:
static IndexResult result;
static IndexResult result2;
```

8.2 Problema de linkeo entre C y C++

Al compilar el proyecto con **g++** pasando todos los archivos juntos (incluyendo los **.c**), el linker no encontraba los símbolos de las funciones C aunque se usaba **extern "C"** en el **.cpp**. El error era: *undefined reference to 'parse_json'*.

El problema era que **g++** aplicaba name mangling de C++ a los archivos **.c**, ignorando el bloque **extern "C"** del **.cpp**. La solución fue compilar los módulos **.c** a archivos **.o** con **gcc** por separado, y luego enlazar todo con **g++**. Esto quedó reflejado en el Makefile con targets separados por archivo.

8.3 Strings con llaves dentro de objetos

La función **find_block_end()** contaba las llaves para encontrar el cierre de un objeto, pero inicialmente no manejaba el caso de strings que contienen los caracteres **{ o }** adentro. Por ejemplo:

```
{"mensaje": "resultado {ok}"}
```

La **}** dentro del string hacía que el contador de profundidad llegara a 0 antes de tiempo, calculando un offset de fin incorrecto. Se corrigió agregando un salto explícito de strings completos dentro del loop de conteo de llaves.

8.4 El \$ del patrón de búsqueda

El patrón llega como **\$/usuarios/[0-9]+/nombre**, donde el **\$** inicial es la convención del proyecto para representar la raíz. Sin embargo, en expresiones regulares POSIX el **\$** es el ancla de fin de línea. Pasarlo directamente a **regcomp()** causaba matches incorrectos. Se solucionó reemplazando el **\$**

inicial por ^ y agregando \$ al final para forzar el match completo.

9. Pruebas Realizadas

9.1 Script de pruebas automático

Se desarrolló un script **test.sh** con 19 casos de prueba organizados en 4 bloques. El script compara la salida del programa contra el resultado esperado y reporta PASS o FAIL para cada caso.

```
bash tests/test.sh

=====
Pruebas de jqindex
=====

--- Bloque 1: datos.json ---
[PASS] build genera el .jnx
[PASS] busqueda nombres usuarios
[PASS] busqueda valor unico string
[PASS] busqueda booleanos
[PASS] busqueda numeros
[PASS] busqueda sin resultados
...
=====
Resultado: 19/19 pruebas pasaron
Todas las pruebas pasaron!
=====
```

9.2 Archivos JSON de prueba

datos.json — Ejemplo del enunciado:

```
{
  "usuarios": [
    {"id": 1, "nombre": "Ana", "activo": true},
    {"id": 2, "nombre": "Luis", "activo": false}
  ],
  "config": {
    "version": "1.0",
    "opciones": {"modo": "auto"}
  }
}
```

simple.json — Objeto plano para prueba básica:

```
{
  "nombre": "Carlos",
  "edad": 30,
  "activo": true,
  "direccion": {"ciudad": "San Jose", "pais": "Costa Rica"}
}
```

complejo.json — Arrays anidados a dos niveles:

```
{
```

```

"empresa": "TEC",
"departamentos": [
  {
    "nombre": "Computo",
    "empleados": [
      {"id": 1, "nombre": "Pedro", "salario": 1500},
      {"id": 2, "nombre": "Laura", "salario": 1800}
    ]
  }
]
}

```

9.3 Tabla de casos de prueba y resultados

Caso	Archivo	Patrón	Esperado	Obtenido
1	datos.json	\$/usuarios/[0-9]+/nombre	["Ana", "Luis"]	PASS
2	datos.json	\$/config/version	"1.0"	PASS
3	datos.json	\$/usuarios/[0-9]+/activo	[true, false]	PASS
4	datos.json	\$/usuarios/[0-9]+/id	[1, 2]	PASS
5	datos.json	\$/noexiste	Sin resultados	PASS
6	simple.json	\$/nombre	"Carlos"	PASS
7	simple.json	\$/edad	30	PASS
8	simple.json	\$/direccion/ciudad	"San Jose"	PASS
9	complejo.json	\$/departamentos/[0-9]+/nombre	["Computo", "Matematica"]	PASS
10	complejo.json	\$/departamentos/0/empleados/[0-9]+/nombre	["Pedro", "Laura"]	PASS

10. Conclusiones

10.1 Lo que se aprendió

El proyecto fue una buena oportunidad para trabajar con varias cosas que no se ven mucho en los cursos anteriores: el manejo directo de offsets en bytes con **fseek**, el uso de **regex.h** de POSIX, y la construcción de un parser desde cero sin apoyarse en librerías externas.

Una de las cosas más interesantes fue entender por qué los sistemas de archivos indexados existen: leer solo el índice y saltar directamente al dato es significativamente más eficiente que releer el archivo completo cada vez, especialmente cuando los archivos son grandes.

También fue útil trabajar con la mezcla de C y C++. Aunque en este proyecto no se usa ninguna característica específica de C++ (el único archivo **.cpp** es básicamente C puro), el proceso de enlazarlo correctamente con los módulos **.c** dejó claro cómo funciona el name mangling y por qué **extern "C"** existe.

10.2 Limitaciones del proyecto

La limitación más obvia es que el parser no maneja todos los casos del estándar JSON. Específicamente, los escapes Unicode (**\u0022**) no se procesan, y strings con saltos de línea o caracteres especiales podrían causar offsets incorrectos en casos muy específicos.

Otra limitación es el tamaño fijo de **MAX_ENTRIES = 10000**. Para archivos JSON muy grandes con decenas de miles de elementos, el índice se truncaría. Una mejora sería usar memoria dinámica con **realloc** para crecer según sea necesario.

El límite de **MAX_PATH_LEN = 512** también podría ser un problema para JSONs muy profundamente anidados, aunque en la práctica es muy difícil llegar a esa profundidad.

10.3 Posibles mejoras futuras

Las mejoras más naturales serían: soporte para actualización incremental del índice (sin tener que regenerarlo completo cuando cambia el JSON), manejo de todos los escapes de strings según el estándar JSON, y soporte para el comando **jqindex get** que extraiga un elemento por path exacto sin regex.

También sería interesante medir el rendimiento comparando el tiempo de búsqueda con índice vs. sin índice en archivos grandes, para cuantificar la mejora real que ofrece el sistema de indexación.